

MÔ PHỎNG CPU 8-CORE VỚI KIẾN TRÚC TẬP LỆNH RISC-V

Multiple Processing System sử dụng Verilog HDL

Thực hiện: Nhóm Học Viên

Báo Cáo Tiến Độ Dự Án Kiến Trúc Máy Tính

Ngày 12 tháng 6 năm 2026

Nội Dung Báo Cáo

- 1 Giới Thiệu Đề Tài
- 2 Kiến Trúc Tập Lệnh (ISA)
- 3 Thiết Kế Hệ Thống (Design)
- 4 Chiến Lược Kiểm Thử (Testing)

1. Giới Thiệu & Lý Do Chọn Đề Tài

Xu thế Đa Nhân (Multi-core Processing)

Giới hạn vật lý về tần số xung nhịp buộc kiến trúc máy tính hiện đại phải chuyển dịch sang kiến trúc đa nhân nhằm tối ưu hóa hiệu năng tính toán song song.

Tại sao chọn RISC-V?

- Kiến trúc tập lệnh (ISA) dạng mở, tinh gọn, có tính ứng dụng cao.
- Cấu trúc định dạng lệnh cố định, tạo điều kiện thuận lợi cho việc thiết kế phần cứng bằng ngôn ngữ mô tả phần cứng (Verilog HDL).

2. Kiến Trúc Tập Lệnh RISC-V 32-bit (RV32I)

Đặc điểm cốt lõi của RV32I:

- Không gian địa chỉ và độ dài lệnh cố định **32-bit**.
- Cơ chế **Load-Store**: Chỉ có lệnh Load/Store mới được truy cập Memory; các phép toán toán học chỉ thực thi trên Tập thanh ghi.
- **32 Thanh ghi đa năng** ($x0 \rightarrow x31$), trong đó $x0$ luôn cố định bằng số 0.
- 6 kiểu định dạng lệnh cơ bản: R, I, S, B, U, J.

Định dạng mã máy đồng nhất

Các trường nguồn dữ liệu ($rs1, rs2$) và đích (rd) luôn nằm ở các vị trí bit cố định → Giúp khối **Decode** phần cứng cực kỳ đơn giản và nhanh chóng.

3. Thiết Kế Hệ Thống: Cấu Trúc Tổng Thể

Sơ đồ phân rã các Module trong mã nguồn Verilog

Hệ thống được chia thành 4 phân vùng chính để quản lý và hiện thực hóa trong mã nguồn:

- ❶ **Datapath (Single Core):** Trục xương sống dịch chuyển và biến đổi dữ liệu.
- ❷ **Control Unit:** Bộ não điều khiển luồng dữ liệu chạy nhịp nhàng qua Pipeline.
- ❸ **Pipeline Hazard Unit:** Khôi cứu nguy và xử lý các xung đột phần cứng.
- ❹ **Interconnect:** Hệ thống trục Bus kết nối, phân xử quyền hạn giữa 8 lõi CPU.

3.1 Xây Dựng Khối Datapath (datapath.v)

Sự khác biệt cốt lõi: Single-Cycle vs Pipeline Datapath

Pipeline Datapath bắt buộc phải tái cấu trúc hoàn toàn vật lý chứ không chỉ khác biệt ở Control Unit.

- **Pipeline Registers:** Chèn 4 hàng thanh ghi đệm (IF/ID, ID/EX, EX/MEM, MEM/WB) để chia cắt Datapath thành 5 khúc biệt lập.
- **Dịch chuyển tín hiệu:** Các tín hiệu điều khiển không chạy trực tiếp mà phải xếp hàng, "đi chung hành trình" dọc theo các thanh ghi đệm này cùng với lệnh.
- **Mở rộng phần cứng:** Bổ sung các bộ Mux lớn tại ngõ vào ALU để sẵn sàng nhận dữ liệu hồi tiếp (Forwarding).

3.2 Control Unit & 5 Giai Đoạn Pipeline (control_unit.v)

Điều khiển luồng dữ liệu chạy gôi đầu nhau qua từng chu kỳ clock trên 5 tầng:

1. Fetch (IF)

Lấy mã lệnh từ Instruction Memory dựa trên giá trị bộ đếm PC.

3. Execute (EX)

Bộ **ALU** thực thi các phép toán số học hoặc tính toán địa chỉ rẽ nhánh.

2. Decode (ID)

Giải mã lệnh sinh tín hiệu điều khiển; Đọc dữ liệu từ Register File.

4. Memory (MEM)

Đọc/Ghi dữ liệu vào Data Cache / RAM nếu gặp lệnh Load/Store.

5. Write Back (WB)

Ghi kết quả tính toán cuối cùng từ RAM/ALU ngược về Register File.

3.3 Quản Lý Xung Đột (pipeline_hazard.v)

Chạy gô đầu lệnh tạo ra các xung đột phần cứng nguy hiểm cần mạch Logic xử lý:

1. Data Hazard (Xung đột dữ liệu)

- Xảy ra khi lệnh sau cần đọc thanh ghi mà lệnh trước chưa kịp ghi xong.
- **Giải pháp:** Thiết kế mạch **Forwarding Unit** để bơm tất dữ liệu từ ngõ ra ALU tầng sau về ngay ngõ vào ALU tầng trước mà không cần đợi.
- Với lệnh Load, thực hiện **Stall** (hoãn) pipeline 1 chu kỳ.

2. Control Hazard (Xung đột điều khiển)

- Xảy ra khi lệnh rẽ nhánh (BEQ, BNE) làm thay đổi dòng chảy của PC.
- **Giải pháp:** Thực hiện **Flush** (xóa bỏ dữ liệu lỗi bằng cách nạp mã lệnh rỗng NOP) tại các tầng nạp sai lệnh trước đó.

3.4 Kết Nối Đa Nhân (interconnect.v)

Để liên kết **8 lõi CPU độc lập** tương tác với hệ thống Bộ nhớ dùng chung (Shared Memory):

- **Bus Interconnect:** Hệ thống dây dẫn cấu trúc hóa kết nối song song dữ liệu và địa chỉ.
- **Bus Arbiter (Bộ trọng tài):** Điều phối quyền truy cập Bus bằng thuật toán phân phối (ví dụ *Round-Robin*), ngăn chặn hiện tượng tranh chấp dữ liệu (Collision) và "đói" tài nguyên bộ nhớ giữa các Core.
- Khi Core A chiếm giữ Bus qua tín hiệu $grant = 1$, các Core khác tự động giữ yêu cầu $req = 1$ và chuyển trạng thái chờ đợi.

4. Chiến Lược Kiểm Thử & Đọc Waveform

Hệ thống được kiểm thử bài bản từ cấp độ Module cô lập đến chạy ứng dụng thực tế:

Cấp độ kiểm thử	Cách thực hiện kiểm chứng trên Waveform	Trạng thái
1. Single Core	Nạp file Hex mẫu. Xem PC tăng tiến và các toán hạng ALU khớp.	Hoàn thành
2. Pipeline Hazard	Gây lỗi đọc-ghi liên tiếp. Kiểm tra tín hiệu ForwardA/B bật lên.	Hoàn thành
3. Multi-Core	Cho 8 Core cùng chạy, theo dõi tín hiệu phân phối grant của Arbiter.	Đang triển khai

Minh chứng Thuyết trình (Bằng chứng trực quan)

Trong báo cáo chi tiết, chúng em sử dụng ảnh chụp Waveform từ công cụ mô phỏng để giải thích rõ ràng các pha **Cache Miss / Stall** và cách hệ thống tự động sửa sai trên dòng xung nhịp thực tế.